Prince Sultan University

College of Computer and Information Sciences / Department of Computer Science

Term 261

CS102 Computer Programming II

RiyadhPoly Project — Overview

Read this overview first. Each phase then has its own document with exactly what to build and test. All three phase documents are published now, so you may work ahead.

Contents

RiyadhPoly Project — Overview 1

The game 2

At a glance 3

Timeline 4

Class design 5

What a game looks like 7

Using AI tools: use them wisely, own your code 8

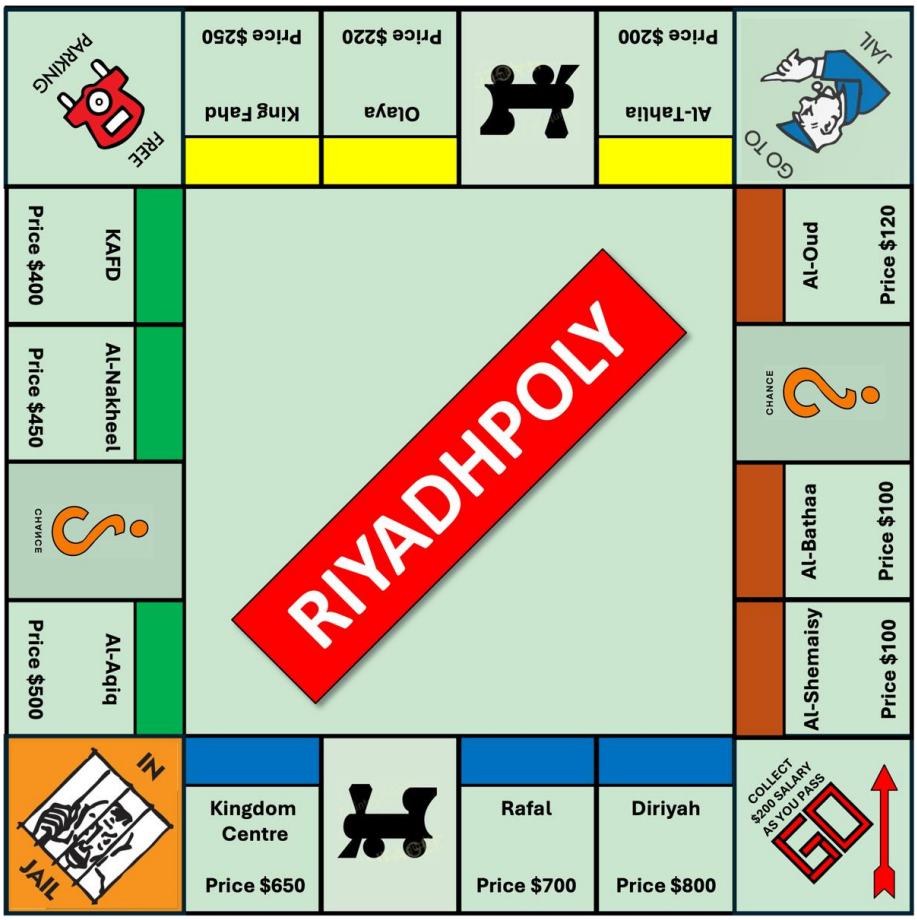
Viva and grading 9

The game

You will build RiyadhPoly, a console version of Monopoly played on a 20-square board of Riyadh landmarks. Four computer-controlled players (bots) start on GO with $800 and take turns in the order they were added, rolling two dice, moving counter-clockwise around the board.

The board is a loop of 20 squares, numbered 0 to 19. After square 19, players continue from square 0 (GO) again.

- **12 properties**, named after Riyadh districts and landmarks. They are grouped into 4 **color groups** of 3: Brown (cheapest), Yellow, Green and Blue (most expensive). Each property has a price and a rent.
- **2 train stations.** They work like a small group of 2.
- **6 special squares:** GO (square 0), two Chance squares, Go To Jail, Park and Jail. These cannot be bought.
- **Buy and collect rent.** When a player lands on an unowned property or station, they can buy it. When they land on a square another player owns, they pay that player rent. Rent doubles if the owner has every square in that color group.
- **Buy-out.** A player who owns all but one square of a group, and lands on that last square owned by someone else, buys it from them at 1.5 × the price.
- **Chance and jail.** A Chance square draws a card that gives money, takes money or moves you. Go To Jail sends you to Jail, where you miss a turn.
- **Salary.** Every time you pass GO you collect $100.
- **Winning.** A player whose cash drops below zero is out. After 20 rounds, the player with the highest net worth (cash plus the value of their properties) wins.



At a glance

The project is worth **10 marks**, done in groups of **exactly 3 students** over **4 weeks**, followed by an **individual viva**. Use Java with any IDE or editor.

What you build, phase by phase:

Phase	You build	The game can now…
1. Design and board	The `Square` classes, `Ownable`, `ColorGroup`, `Board`, `InvalidBoardException`	load `board.txt` and print the board
2. Players and money	`Player` and two bots, `Dice`, buying, rent, the buy-out rule, a simple round loop, the leaderboard, `RentTest`	play 20 rounds of buying and paying rent

Phase	You build	The game can now…
3. Full game	Chance cards, jail, bankruptcy with an `Iterator`, final standings	play a complete game and name a winner

The five rules that matter most:

1. A bot buys an unowned square it lands on if its buying rule says yes.
2. Landing on someone else's square costs rent, **doubled** if they own the whole color group.
3. **Buy-out:** owning every other square of a group lets you buy the last one from its owner at 1.5 × the price.
4. **Jail:** you skip one turn.
5. **Bankruptcy:** cash below $0 means you are out, and your squares go back to the bank.

Your documents and files (all on the LMS):

- This overview
- Phase 1: Design and Board, Phase 2: Players and Money, Phase 3: Full Game
- `board.txt` (the default board; your group also receives its own board file) and `RiyadhPoly_sample_output.txt` (a full 20-round sample game)

Timeline

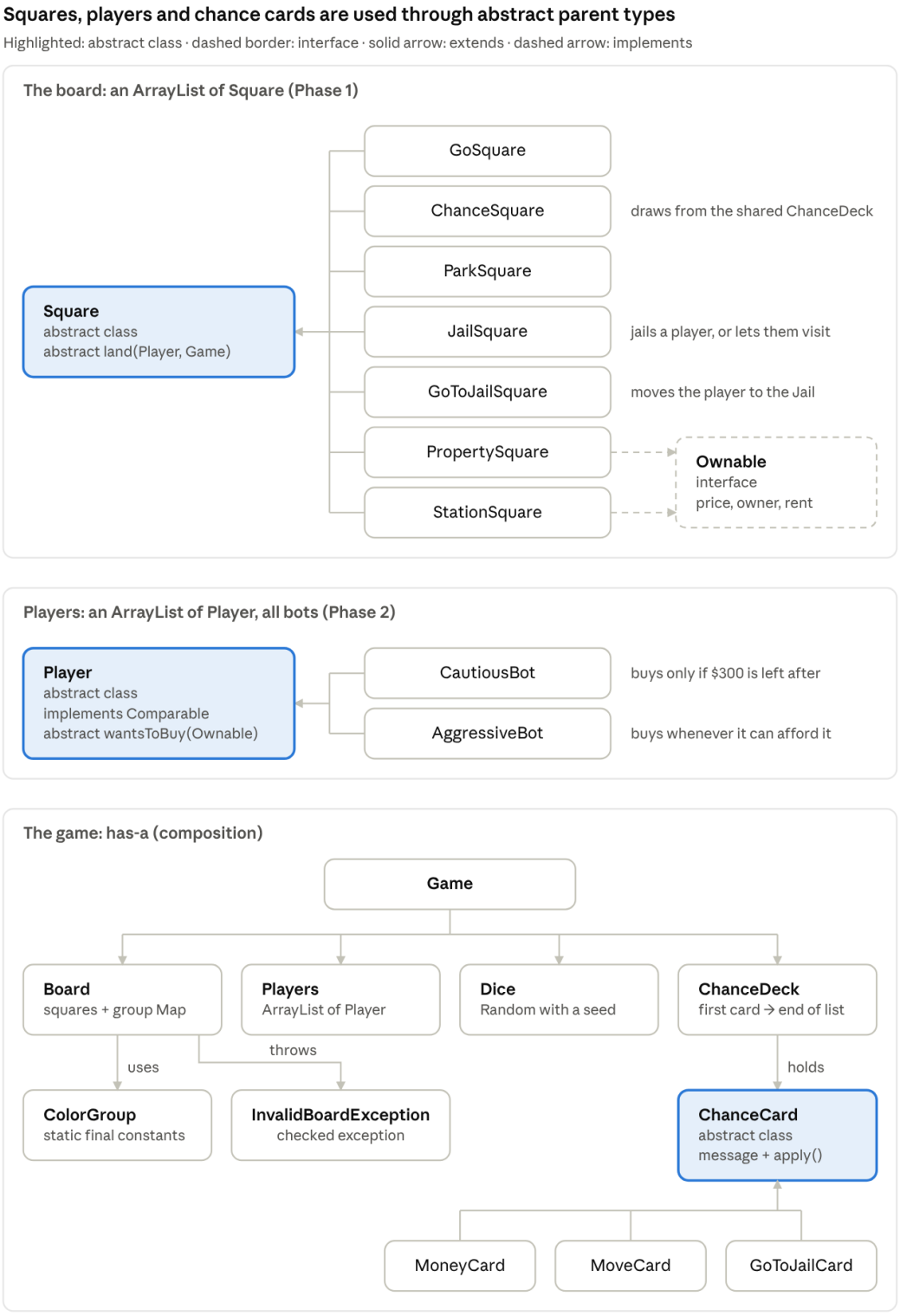
Submit each phase on the LMS by its deadline. Phases 1 and 2 are followed by a **5-minute check-in in the next lab session**: your group runs its code, and the instructor asks **each member** one short question. Work is never thrown away: if a phase had problems, fix them in the next one. Only the final submission is marked for correctness.

Phase	Deadline	Marks
0. Form groups: exactly 3 students, joined on the LMS. Students without a group are placed in one by the instructor	Oct 15, 2026	—
1. Design and board	Oct 24, 2026	2 (check-in)
2. Players and money	Oct 31, 2026	2 (check-in)

Phase	Deadline	Marks
3. Full game (final submission)	Nov 4, 2026	5.5 (final code) + 0.5 (AI log)
Viva (individual)	Nov 8, 2026 and Nov 9, 2026	Sets the % of the 10 marks you keep

Class design

Your program must use the classes below; you may add helper classes and methods. The game code works with `Square`, `Player` and `ChanceCard` only, never with a specific subclass: each object decides its own behavior. Each phase document says which classes to build and gives code hints.



required class design · 3 groups

Java concepts and where they must appear

Concept	Where in the project	Phase
Abstract class, inheritance	`Square` and its 7 subclasses; `Player`, `CautiousBot`, `AggressiveBot`; `ChanceCard` and its subclasses	1, 2, 3
Polymorphism	`square.land(player, game)`, `player.wantsToBuy(ownable)`, `card.apply(player, game)`	1, 2, 3
Interface	`Ownable`, implemented by `PropertySquare` and `StationSquare`	1
Static final constants	Color groups in a `ColorGroup` class; every game number (`INITIAL_CASH`, `SALARY`, …)	1, 2
ArrayList, Iterator	The board, the players, the chance deck; removing bankrupt players	1, 2, 3
Map	`Board` keeps `Map<String, ArrayList<Ownable>>`: each color group and its squares	1
Comparable	`Player implements Comparable<Player>` by net worth, for the leaderboard	2
File I/O, exceptions	Reading `board.txt`; `InvalidBoardException`; safe input with `Scanner`	1, 2

What a game looks like

The game runs by itself once the seed, names and bot types are entered. Below are rounds 1 and 4 of a sample game: in round 4, Amal buys out Sara's Al-Oud and then collects double rent on the brown group. Amal and Huda are aggressive bots; Sara and Noura are cautious. The full 20-round game, including a bankruptcy, is in `RiyadhPoly_sample_output.txt`. Dice values are for illustration only.

```
**********************************************
**             Welcome to RiyadhPoly            **
**********************************************
Board loaded: 20 squares.
Seed (press Enter for random): 42
Player 1 name: Amal
Player 1 type (C = cautious, A = aggressive): A
Player 2 name: Sara
Player 2 type (C = cautious, A = aggressive): C
Player 3 name: Noura
```

```
Player 3 type (C = cautious, A = aggressive): C
Player 4 name: Huda
Player 4 type (C = cautious, A = aggressive): A
Playing order: Amal -> Sara -> Noura -> Huda

========== ROUND 1 ==========
Amal's turn. Rolled 1 + 1 = 2. Moving to square 2.
Amal landed on Al-Bathaa (BROWN). Unowned, price $100.
Amal bought Al-Bathaa for $100.
Sara's turn. Rolled 1 + 3 = 4. Moving to square 4.
Sara landed on Al-Oud (BROWN). Unowned, price $120.
Sara bought Al-Oud for $120.
Noura's turn. Rolled 2 + 4 = 6. Moving to square 6.
Noura landed on Al-Tahlia (YELLOW). Unowned, price $200.
Noura bought Al-Tahlia for $200.
Huda's turn. Rolled 4 + 5 = 9. Moving to square 9.
Huda landed on King Fahd (YELLOW). Unowned, price $250.
Huda bought King Fahd for $250.
---------- ROUND 1 LEADERBOARD ----------
1. Amal    cash $700   net worth $800
2. Sara    cash $680   net worth $800
3. Noura   cash $600   net worth $800
4. Huda    cash $550   net worth $800

   ... rounds 2 and 3: Amal also buys Al-Shemaisy, Huda goes to jail ...

========== ROUND 4 ==========
Amal's turn. Rolled 1 + 2 = 3. Moving to square 4.
Amal landed on Al-Oud (BROWN). Owned by Sara.
Amal owns the rest of BROWN and buys Al-Oud from Sara for $180.
Sara's turn. Rolled 1 + 3 = 4. Moving to square 19.
Sara landed on Diriyah (BLUE). Unowned, price $800.
Sara does not buy Diriyah.
Noura's turn. Rolled 2 + 4 = 6. Moving to square 2.
Noura passed GO and collected $100.
Noura landed on Al-Bathaa (BROWN). Owned by Amal (whole group). Rent: $96.
Noura paid $96 to Amal.
Huda is in jail. Turn skipped.
---------- ROUND 4 LEADERBOARD ----------
1. Huda    cash $778   net worth $1028
2. Amal    cash $666   net worth $986
3. Sara    cash $640   net worth $860
4. Noura   cash $476   net worth $676

   ... rounds 5 to 20: see RiyadhPoly_sample_output.txt ...
```

Using AI tools: use them wisely, own your code

You may use AI tools (ChatGPT, Claude, Copilot, Gemini and others) in this project. Using AI is not cheating. **Submitting code you cannot explain is.** In the viva, you will explain and change your code without AI, so every line you submit must be one you understand.

Good ways to use AI

- Ask it to **explain** a concept: "What is the difference between an abstract class and an interface?"
- Ask it to **explain an error message** or an exception you got, then fix it yourself.
- **Write your code first**, then ask for a review: "What could go wrong in this method?"
- Ask it to **quiz you** on your own code before the viva.

Uses that will hurt you

- Pasting a phase document and submitting the generated program.
- Generating whole classes and then not reading them.
- Letting AI "fix" a bug without understanding what was wrong.

Be careful: AI gets this project wrong. AI tools often make the same mistakes in this kind of program: removing players from a list while looping over it, using `%` with a negative number, and updating only one side when a square changes owner. If you used AI and did not check its code, the viva will show it.

AI-use log (required, 0.5 marks). Each group keeps a short log and submits it with the final code. Write one row each time AI meaningfully helped you. An honest log with many rows is fine. A missing or empty log while the code shows clear signs of AI use is not.

Date	Student	Tool	What I asked	What I used from the answer	What I changed or learned
e.g. 12 Oct	Sara	ChatGPT	Why do I get ConcurrentModificationException?	The idea of using an Iterator	Rewrote the bankruptcy check with iterator.remove()

Rule of thumb: if you cannot explain a line to your instructor, it is not your line yet. Ask the AI to explain it until you can, or rewrite it.

Viva and grading

Viva. On 8 and 9 November, each student has a **10-minute viva on their group's final code**, alone, on a lab machine, with no internet or AI. You may be asked about **any** part of the code,

not only the part you wrote. The best way to prepare is to understand every class in your group's code.

Part	Time	What happens	Points
A. Explain	4 min	3 questions about how your code works	6 (2 each)
B. Change	6 min	1 small change to your code: make it, run it, explain it	4 (2 working, 2 explanation)
Viva score			10

The viva decides how much of your mark you keep. The viva itself earns no marks. Instead, your viva score sets the percentage of your project mark (out of 10) that becomes your final mark:

Viva score	9 to 10	7 to 8.5	5 to 6.5	3 to 4.5	0 to 2.5
You keep	100%	90%	75%	50%	25%

Example: your project mark is 8.5 out of 10 and your viva score is 6. You keep 75%, so your final mark is 8.5 × 0.75 = 6.4. Code you cannot explain earns little.

Grading rubric. The 10 marks come from the check-ins, the final code and the AI-use log. Your **final mark = project mark × your viva percentage**.

Component	Group/individual	What earns full marks	Marks
Phase 1 check-in	Group 1.5 + individual 0.5	Diagram matches the code; the board loads, prints, and rejects a broken file. Each member answers their question correctly	2
Phase 2 check-in	Group 1.5 + individual 0.5	A full 20-round game runs with the given seed; `RentTest` shows a buy-out and rent doubling. Each member answers correctly	2

Component	Group/individual	What earns full marks	Marks
Final code: rules	Group	Every rule works, including the chance deck, jail, the buy-out rule, bankruptcy, and the winner	3
Final code: design	Group	Uses every concept in the Java concepts table where it belongs	1.5
Final code: quality	Group	Constants instead of magic numbers; input validated; clear names; short comments on non-obvious code	1
AI-use log	Group	Honest, specific rows that match the code	0.5
Project mark			10
Viva	Individual	Sets the percentage of the project mark you keep	